

Rbtree

红黑树

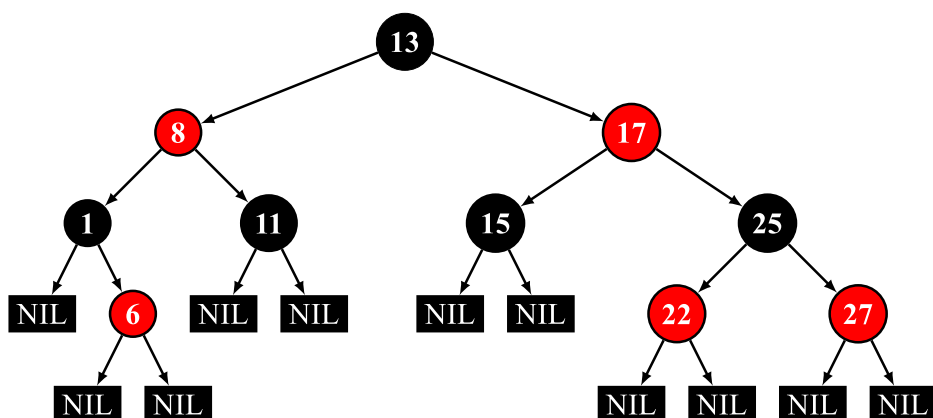
红黑树是一种自平衡的二叉搜索树。每个节点额外存储了一个 color 字段 ("RED" or "BLACK"), 用于确保树在插入和删除时保持平衡。

性质

一棵合法的红黑树必须遵循以下四条性质：

1. 节点为红色或黑色
2. NIL 节点（空叶子节点）为黑色
3. 红色节点的子节点为黑色
4. 从根节点到 NIL 节点的每条路径上的黑色节点数量相同

下图为一棵合法的红黑树：



注：部分资料中还加入了第五条性质，即根节点必须为黑色，这条性质要求完成插入操作后若根节点为红色则将其染黑，但由于将根节点染黑的操作也可以延迟至删除操作时进行，因此，该条性质并非必须满足。（在本文给出的代码实现中就没有选择满足该性质）。为严谨起见，这里同时引用 [维基百科原文](#) 进行说明：

Some authors, e.g. Cormen & al.,¹ claim "the root is black" as fifth requirement; but not Mehlhorn & Sanders² or Sedgewick & Wayne.³ Since the root can always be changed from red to black, this rule has little effect on analysis. This article also omits it, because it slightly disturbs the recursive algorithms and proofs.

结构

红黑树类的定义

```
1 template <typename Key, typename Value, typename Compare = std::less<Key>>
2 class RBTreeMap {
3     // 排序函数
4     Compare compare = Compare();
5
6     // 节点结构体
7     struct Node {
8         ...
9     };
10
11     // 根节点指针
12     Node* root = nullptr;
13     // 记录红黑树中当前的节点个数
14     size_t count = 0;
15 }
```

节点维护的信息

Identifier	Type	Description
left	Node*	左子节点指针
right	Node*	右子节点指针
parent	Node*	父节点指针
color	enum { BLACK, RED }	颜色枚举
key	Key	节点键值，具有唯一性和可排序性
value	Value	节点内储存的值

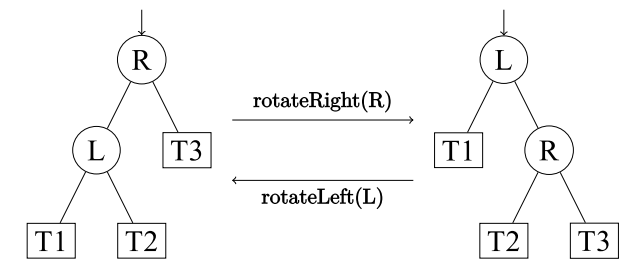
注：由于本文提供的代码示例中使用 `std::shared_ptr` 进行内存管理，对此不熟悉的读者可以将下文中所有的 `NodePtr` 和 `ConstNodePtr` 理解为裸指针 `Node*`。但在实现删除操作时若使用 `Node*` 作为节点引用需注意应手动释放内存以避免内存泄漏，该操作在使用 `std::shared_ptr` 作为节点引用的示例代码中并未体现。

过程

注：由于红黑树是由 B 树衍生而来（发明时的最初的名字 `symmetric binary B-tree` 足以证明这点），并非直接由平衡二叉树外加限制条件推导而来，插入操作的后续维护和删除操作的后续维护中部分对操作的解释作用仅是帮助理解，并不能将其作为该操作的原理推导和证明。

旋转操作

旋转操作是多数平衡树能够维持平衡的关键，它能在不改变一棵合法 BST 中序遍历结果的情况下改变局部节点的深度。



如上图，从左图到右图的过程被称为右旋，右旋操作会使得 子树上结点的深度均加 1，使 子树上结点的深度均减 1，而 子树上节点的深度则不变。从右图到左图的过程被称为左旋，左旋是右旋的镜像操作。

这里给出红黑树中节点的左旋操作的示例代码：

▼ 实现

```
1 void rotateLeft(ConstNodePtr node) {
2     // clang-format off
3     //      |
4     //      N          S
5     //    / \      l-rotate(N)  / \
6     //   L  S  =====>   N  R
7     //      / \          / \
8     //     M  R          L  M
9     assert(node != nullptr && node->right != nullptr);
10    // clang-format on
11    NodePtr parent = node->parent;
12    Direction direction = node->direction();
13
14    NodePtr successor = node->right;
15    node->right = successor->left;
16    successor->left = node;
17
18    // 以下的操作用于维护各个节点的`parent`指针
19    // `Direction`的定义以及`maintainRelationship`
20    // 的实现请参照文章末尾的完整示例代码
21    maintainRelationship(node);
22    maintainRelationship(successor);
23
24    switch (direction) {
25        case Direction::ROOT:
26            this->root = successor;
27            break;
28        case Direction::LEFT:
29            parent->left = successor;
30            break;
31        case Direction::RIGHT:
32            parent->right = successor;
33            break;
34    }
35
36    successor->parent = parent;
37 }
```

注：代码中的 `successor` 并非平衡树中的后继节点，而是表示取代原本节点的新节点，由于在图示中 `replacement` 的简称 `R` 会与右子节点的简称 `R` 冲突，因此此处使用 `successor` 避免歧义。

插入操作

红黑树的插入操作与普通的 BST 类似，对于红黑树来说，新插入的节点初始为红色，完成插入后需根据插入节点及相关节点的状态进行修正以满足上文提到的四条性质。

插入后的平衡维护

Case 1

该树原先为空，插入第一个节点后不需要进行修正。

Case 2

当前的节点的父节点为黑色且为根节点，这时性质已经满足，不需要进行修正。

Case 3

当前节点 `N` 的父节点 `P` 是为根节点且为红色，将其染为黑色即可，此时性质也已满足，不需要进一步修正。

▼ 实现

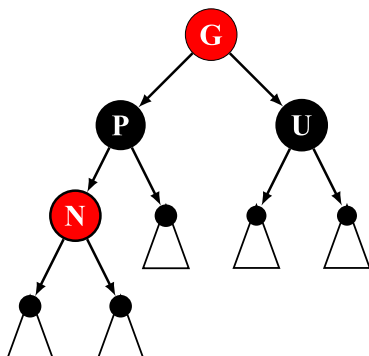
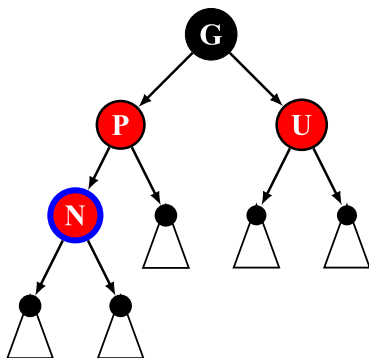
```
1 // clang-format off
2 // Case 3: Parent is root and is RED
3 //   Paint parent to BLACK.
4 //   &LTP>          [P]
5 //   |   =====>  |
6 //   &LTN>          &LTN>
7 //   p.s.
8 //   `&LTX>` is a RED node;
9 //   `[X]` is a BLACK node (or NIL);
10 //   `{X}` is either a RED node or a BLACK node;
11 // clang-format on
12 assert(node->parent->isRed());
13 node->parent->color = Node::BLACK;
14 return;
```

Case 4

当前节点 N 的父节点 P 和叔节点 U 均为红色，此时 P 包含了一个红色子节点，违反了红黑树的性质，需要进行重新染色。由于在当前节点 N 之前该树是一棵合法的红黑树，根据性质 3 可以确定 N 的祖父节点 G 一定是黑色，这时只要后续操作可以保证以 G 为根节点的子树在不违反性质 4 的情况下再递归维护祖父节点 G 以保证性质 3 即可。

因此，这种情况的维护需要：

1. 将 P, U 节点染黑，将 G 节点染红（可以保证每条路径上黑色节点个数不发生改变）。
2. 递归维护 G 节点（因为不确定 G 的父节点的状态，递归维护可以确保性质 3 成立）。



▼ 实现

```

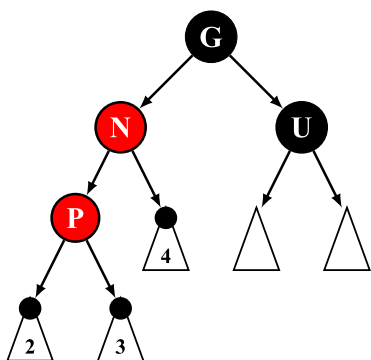
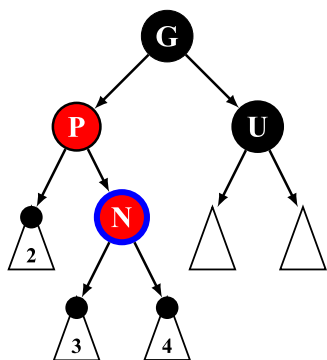
1 // clang-format off
2 // Case 4: Both parent and uncle are RED
3 //   Paint parent and uncle to BLACK;
4 //   Paint grandparent to RED.
5 //       [G]           &LTG>
6 //       / \           / \
7 //   &LTP> &LTU> ==> [P] [U]
8 //       /           /
9 //   &LTN>           &LTN>
10 // clang-format on
11 assert(node->parent->isRed());
12 node->parent->color = Node::BLACK;
13 node->uncle()->color = Node::BLACK;
14 node->grandParent()->color = Node::RED;
15 maintainAfterInsert(node->grandParent());
16 return;

```

Case 5

当前节点 N 与父节点 P 的方向相反（即 N 节点为右子节点且父节点为左子节点，或 N 节点为左子节点且父节点为右子节点。类似 AVL 树中 LR 和 RL 的情况）。根据性质 4，若 N 为新插入节点， U 则为 NIL 黑色节点，否则为普通黑色节点。

该种情况无法直接进行维护，需要通过旋转操作将子树结构调整为 Case 6 的初始状态并进入 Case 6 进行后续维护。



▼ 实现

```

1 // clang-format off
2 // Case 5: Current node is the opposite direction as parent
3 // Step 1. If node is a LEFT child, perform l-rotate to parent;
4 // If node is a RIGHT child, perform r-rotate to parent.
5 // Step 2. Goto Case 6.
6 // [G] [G]
7 // / \ rotate(P) / \
8 // &LTP> [U] =====> &LTN> [U]
9 // \ /
10 // &LTN> &LTP>
11 // clang-format on
12
13 // Step 1: Rotation
14 NodePtr parent = node->parent;
15 if (node->direction() == Direction::LEFT) {
16 rotateRight(node->parent);
17 } else /* node->direction() == Direction::RIGHT */ {
18 rotateLeft(node->parent);
19 }
20 node = parent;
21 // Step 2: vvv

```

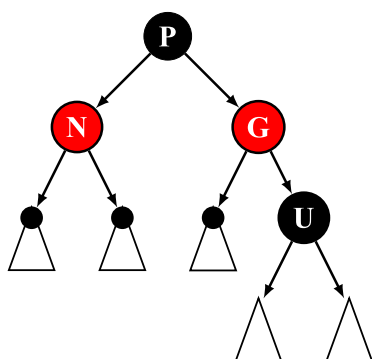
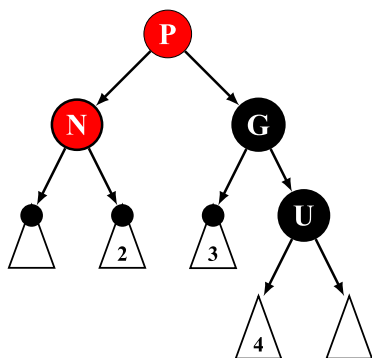
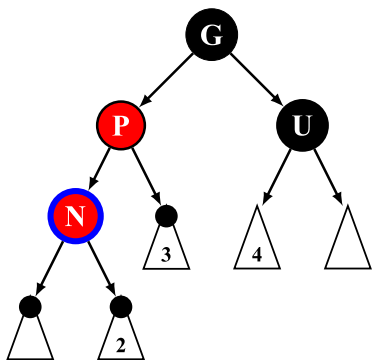
Case 6

当前节点 N 与父节点 P 的方向相同（即 N 节点为右子节点且父节点为右子节点，或 N 节点为左子节点且父节点为左子节点。类似 AVL 树中 LL 和 RR 的情况）。根据性质 4，若 N 为新插入节点，U 则为 NIL 黑色节点，否则为普通黑色节点。

在这种情况下，若想在不变结构的情况下使得子树满足性质 3，则需将 G 染成红色，将 P 染成黑色。但若这样维护的话则性质 4 被打破，且无法保证在 G 节点的父节点上性质 3 是否成立。而选择通过旋转改变子树结构后再进行重新染色即可同时满足性质 3 和 4。

因此，这种情况的维护需要：

1. 若 N 为左子节点则右旋祖父节点 G，否则左旋祖父节点 G。（该操作使得旋转过后 P - N 这条路径上的黑色节点个数比 P - G - U 这条路径上少 1，暂时打破性质 4）。
2. 重新染色，将 P 染黑，将 G 染红，同时满足了性质 3 和 4。



▼ 实现

```

1 // clang-format off
2 // Case 6: Current node is the same direction as parent
3 // Step 1. If node is a LEFT child, perform r-rotate to grandparent;
4 // If node is a RIGHT child, perform l-rotate to grandparent.
5 // Step 2. Paint parent (before rotate) to BLACK;
6 // Paint grandparent (before rotate) to RED.
7 // [G] <--> &LTP> [P]
8 // / \ rotate(G) / \ repaint / \
9 // &LTP> [U] <====> &LTN> [G] <====> &LTN> &LTG>
10 // / \
11 // &LTN> [U] [U]
12 // clang-format on
13 assert(node->grandParent() != nullptr);
14
15 // Step 1
16 if (node->parent->direction() == Direction::LEFT) {
17 rotateRight(node->grandParent());
18 } else {
19 rotateLeft(node->grandParent());
20 }
21
22 // Step 2
23 node->parent->color = Node::BLACK;
24 node->sibling()->color = Node::RED;
25
26 return;

```

删除操作

红黑树的删除操作情况繁多，较为复杂。这部分内容主要通过代码示例来进行讲解。大多数红黑树的实现选择将节点的删除以及删除之后的维护写在同一个函数或逻辑块中（例如 [Wikipedia](#) 给出的 [代码示例](#)，[linux 内核中的 rbtree](#) 以及 GNU libstdc++ 中的 [std::Rb_tree](#) 都使用了类似的写法）。笔者则认为这种实现方式并不利于对算法本身的理解，因此，本文给出的示例代码参考了 OpenJDK 中 [TreeMap](#) 的实现，将删除操作本身与删除后的平衡维护操作解耦成两个独立的函数，并对这两部分的逻辑单独进行分析。

Case 0

若待删除节点为树中唯一的节点的话，直接删除即可，这里不将其算作删除操作的 3 种基本情况中。

Case 1

若待删除节点 N 既有左子节点又有右子节点，则需找到它的前驱或后继节点进行替换（仅替换数据，不改变节点颜色和内部引用关系），则后续操作中只需要将后继节点删除即可。这部分操作与普通 BST 完全相同，在此不再过多赘述。

注：这里选择的前驱或后继节点保证不会是一个既有非 NIL 左子节点又有非 NIL 右子节点的节点。这里拿后继节点进行简单说明：若该节点包含非空左子节点，则该节点并非是 N 节点右子树上键值最小的节点，与后继节点的性质矛盾，因此后继节点的左子节点必须为 NIL。

▼ 实现


```

1 // clang-format off
2 // Case 1: If the node is strictly internal
3 // Step 1. Find the successor S with the smallest key
4 // and its parent P on the right subtree.
5 // Step 2. Swap the data (key and value) of S and N,
6 // S is the node that will be deleted in place of N.
7 // Step 3. N = S, goto Case 2, 3
8 //      |
9 //      N          S
10 //    / \        / \
11 //   L  .. swap(N, S) L  ..
12 //      | =====>  |
13 //      P          P
14 //    / \        / \
15 //   S  ..      N  ..
16 // clang-format on
17
18 // Step 1
19 NodePtr successor = node->right;
20 NodePtr parent = node;
21 while (successor->left != nullptr) {
22     parent = successor;
23     successor = parent->left;
24 }
25 // Step 2
26 swapNode(node, successor);
27 maintainRelationship(parent);
28 // Step 3: vvv

```

Case 2

待删除节点为叶子节点，若该节点为红色，直接删除即可，删除后仍能保证红黑树的 4 条性质。若为黑色，删除后性质 4 被打破，需要重新进行维护。

注：由于维护操作不会改变待删除节点的任何结构和数据，因此此处的代码示例中为了实现方便起见选择先进行维护，再解引用相关节点。

▼ 实现

```

1 // clang-format off
2 // Case 2: Current node is a leaf
3 // Step 1. Unlink and remove it.
4 // Step 2. If N is BLACK, maintain N;
5 // If N is RED, do nothing.
6 // clang-format on
7 // The maintain operation won't change the node itself,
8 // so we can perform maintain operation before unlink the node.
9 if (node->isBlack()) {
10     maintainAfterRemove(node);
11 }
12 if (node->direction() == Direction::LEFT) {
13     node->parent->left = nullptr;
14 } else /* node->direction() == Direction::RIGHT */ {
15     node->parent->right = nullptr;
16 }

```

Case 3

待删除节点 N 有且仅有一个非 NIL 子节点，则子节点 S 一定为红色。因为如果子节点 S 为黑色，则 S 的黑深度和待删除结点的黑深度不同，违反性质 4。由于子节点 S 为红色，则待删除节点 N 为黑色，直接使用子节点 S 替代 N 并将其染黑后即可满足性质 4。

▼ 实现

```
1 // Case 3: Current node has a single left or right child
2 //   Step 1. Replace N with its child
3 //   Step 2. Paint N to BLACK
4 NodePtr parent = node->parent;
5 NodePtr replacement = (node->left != nullptr ? node->left : node->right);
6
7 switch (node->direction()) {
8     case Direction::ROOT:
9         this->root = replacement;
10        break;
11    case Direction::LEFT:
12        parent->left = replacement;
13        break;
14    case Direction::RIGHT:
15        parent->right = replacement;
16        break;
17 }
18
19 if (!node->isRoot()) {
20     replacement->parent = parent;
21 }
22
23 node->color = Node::BLACK;
```

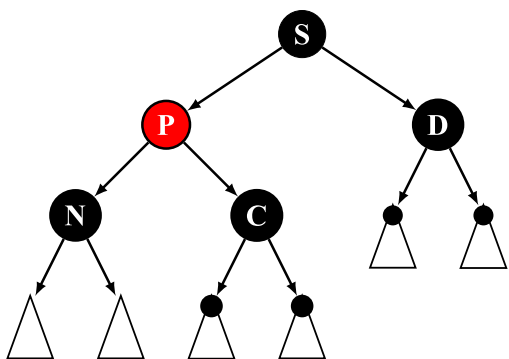
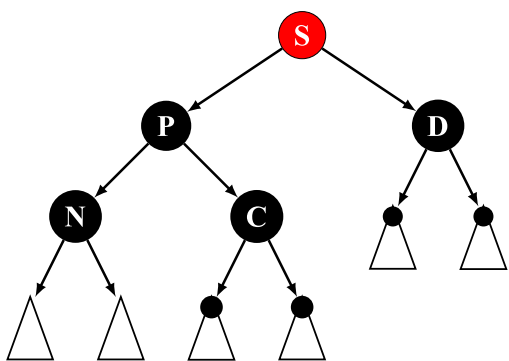
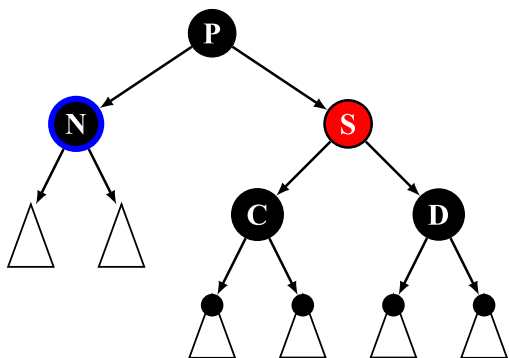
删除后的平衡维护

Case 1

兄弟节点 (sibling node) S 为红色，则父节点 P 和侄节点 (nephew node) C 和 D 必为黑色（否则违反性质 3）。与插入后维护操作的 Case 5 类似，这种情况下无法通过直接的旋转或染色操作使其满足所有性质，因此通过前置操作优先保证部分结构满足性质，再进行后续维护即可。

这种情况的维护需要：

1. 若待删除节点 N 为左子节点，左旋 P；若为右子节点，右旋 P。
2. 将 S 染黑，P 染红（保证 S 节点的父节点满足性质 4）。
3. 此时只需根据结构，在以 P 节点为根的子树中，继续对节点 N 进行维护即可（无需再考虑旋转染色后的 S 和 D 节点）。



▼ 实现

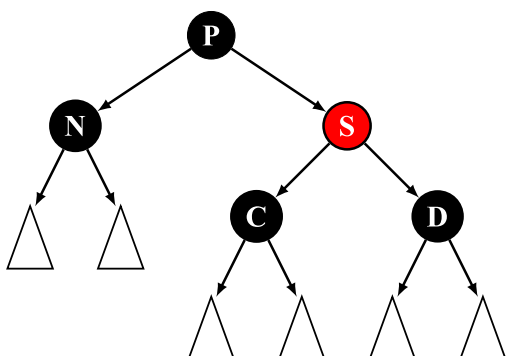
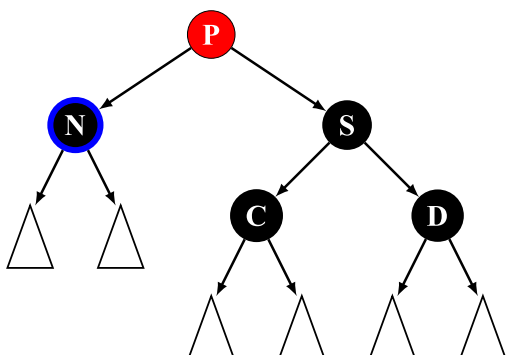
```

1 // clang-format off
2
3 // Case 1: Sibling is RED, parent and nephews must be BLACK
4 //   Step 1. If N is a left child, left rotate P;
5 //           If N is a right child, right rotate P.
6 //   Step 2. Paint S to BLACK, P to RED
7 //   Step 3. Goto Case 2, 3, 4, 5
8 //       [P]                &LTS>                [S]
9 //       / \    l-rotate(P)  / \    repaint    / \
10 //    [N] &LTS> =====> [P] [D] =====> &LTP> [D]
11 //       / \                / \                / \
12 //      [C] [D]            [N] [C]            [N] [C]
13 // clang-format on
14 ConstNodePtr parent = node->parent;
15 assert(parent != nullptr && parent->isBlack());
16 assert(sibling->left != nullptr && sibling->left->isBlack());
17 assert(sibling->right != nullptr && sibling->right->isBlack());
18 // Step 1
19 rotateSameDirection(node->parent, direction);
20 // Step 2
21 sibling->color = Node::BLACK;
22 parent->color = Node::RED;
23 // Update sibling after rotation
24 sibling = node->sibling();
25 // Step 3: vvv

```

Case 2

兄弟节点 S 和侄节点 C, D 均为黑色，父节点 P 为红色。此时只需将 S 染红，将 P 染黑即可满足性质 3 和 4。



▼ 实现

```

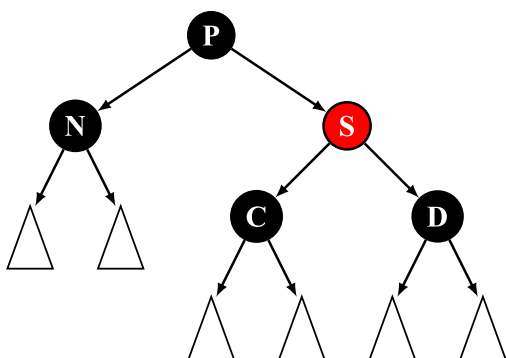
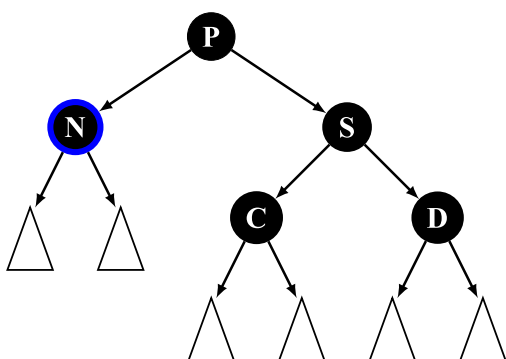
1 // clang-format off
2 // Case 2: Sibling and nephews are BLACK, parent is RED
3 //   Swap the color of P and S
4 //       &LTP>           [P]
5 //       / \           / \
6 //   [N] [S]  =====> [N] &LTS>
7 //       / \           / \
8 //   [C] [D]           [C] [D]
9 // clang-format on
10 sibling->color = Node::RED;
11 node->parent->color = Node::BLACK;
12 return;

```

Case 3

兄弟节点 S，父节点 P 以及侄节点 C, D 均为黑色。

此时也无法通过一步操作同时满足性质 3 和 4，因此选择将 S 染红，优先满足局部性质 4 的成立，再递归维护 P 节点根据上部结构进行后续维护。



▼ 实现

```

1 // clang-format off
2 // Case 3: Sibling, parent and nephews are all black
3 //   Step 1. Paint S to RED
4 //   Step 2. Recursively maintain P
5 //       [P]           [P]
6 //       / \           / \
7 //   [N] [S]  =====> [N] &LTS>
8 //       / \           / \
9 //   [C] [D]           [C] [D]
10 // clang-format on
11 sibling->color = Node::RED;
12 maintainAfterRemove(node->parent);
13 return;

```

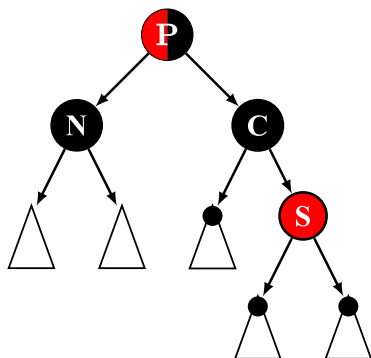
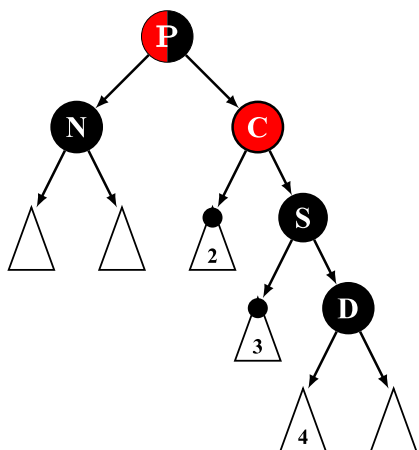
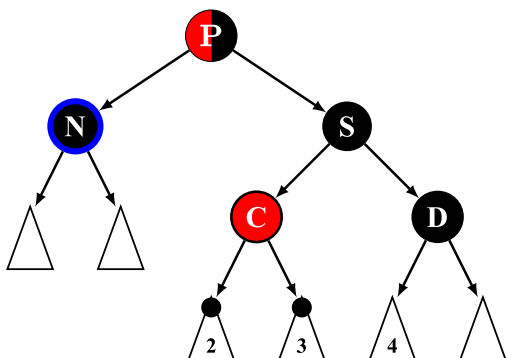
Case 4

兄弟节点是黑色，且与 N 同向的侄节点 C（由于没有固定中文翻译，下文还是统一将其称作 close nephew）为红色，与 N 反向的侄节点 D（同理，下文称作 distant nephew）为黑色，父节点既可为红色又可为黑色。

此时同样无法通过一步操作使其满足性质，因此优先选择将其转变为 Case 5 的状态利用后续 Case 5 的维护过程进行修正。

该过程分为三步：

1. 若 N 为左子节点，右旋 S，否则左旋 S。
2. 将节点 S 染红，将节点 C 染黑。
3. 此时已满足 Case 5 的条件，进入 Case 5 完成后续维护。



▼ 实现

```

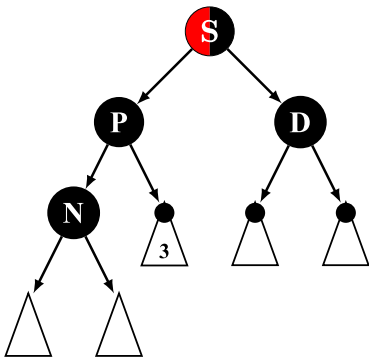
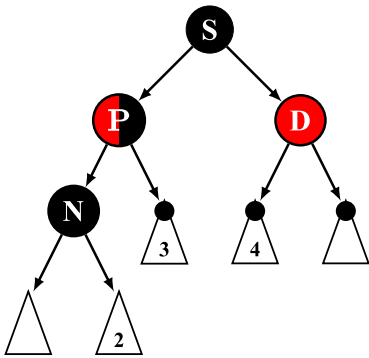
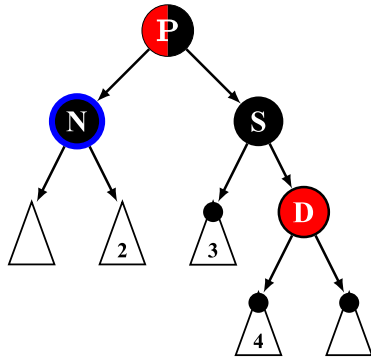
1 // clang-format off
2 // Case 4: Sibling is BLACK, close nephew is RED,
3 //         distant nephew is BLACK
4 // Step 1. If N is a left child, right rotate S;
5 //         If N is a right child, left rotate S.
6 // Step 2. Swap the color of close nephew and sibling
7 // Step 3. Goto case 5
8 //
9 //         {P}           {P}
10 //      {P}      / \      / \
11 //      / \    r-rotate(S) [N] &LTC> repaint [N] [C]
12 //      [N] [S] =====> \ =====> \
13 //      / \                [S]          &LTS>
14 //      &LTC> [D]          \            \
15 //                        [D]          [D]
16
17 // clang-format on
18
19 // Step 1
20 rotateOppositeDirection(sibling, direction);
21 // Step 2
22 closeNephew->color = Node::BLACK;
23 sibling->color = Node::RED;
24 // Update sibling and nephews after rotation
25 sibling = node->sibling();
26 closeNephew = direction == Direction::LEFT ? sibling->left : sibling->right;
27 distantNephew = direction == Direction::LEFT ? sibling->right : sibling->left;
28 // Step 3: vvv

```

Case 5

兄弟节点是黑色，且 distant nephew 节点 D 为红色，close nephew 节点和父节点既可为红色又可为黑色。此时性质 4 无法满足，通过旋转操作使得黑色节点 S 变为该子树的根节点再进行染色即可满足性质 4。具体步骤如下：

1. 若 N 为左子节点，左旋 P，反之右旋 P。
2. 交换父节点 P 和兄弟节点 S 的颜色，此时性质 3 可能被打破。
3. 将 distant nephew 节点 D 染黑，同时保证了性质 3 和 4。



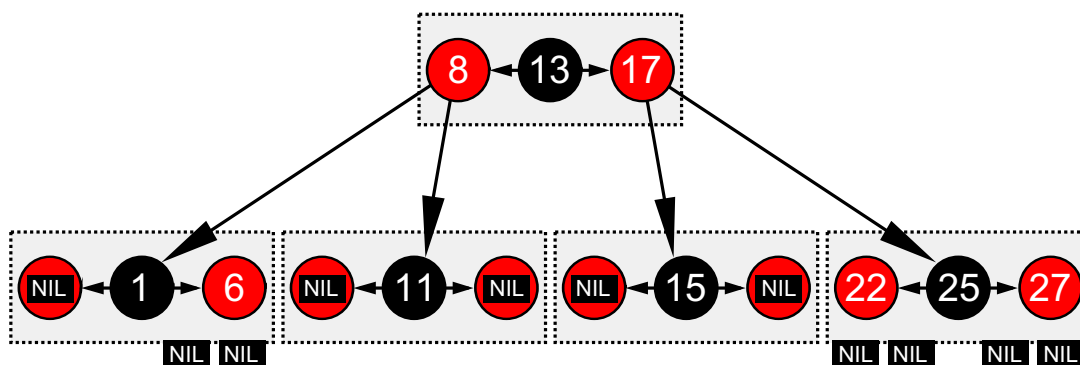
▼ 实现

```

1 // clang-format off
2 // Case 5: Sibling is BLACK, distant nephew is RED
3 //   Step 1. If N is a left child, left rotate P;
4 //           If N is a right child, right rotate P.
5 //   Step 2. Swap the color of parent and sibling.
6 //   Step 3. Paint distant nephew D to BLACK.
7 //           {P}           [S]           {S}
8 //           / \   1-rotate(P) / \   repaint / \
9 //           [N] [S] =====> {P} &LTD> =====> [P] [D]
10 //           / \           / \           / \
11 //           {C} &LTD>      [N] {C}      [N] {C}
12 // clang-format on
13 assert(distantNephew->isRed());
14 // Step 1
15 rotateSameDirection(node->parent, direction);
16 // Step 2
17 sibling->color = node->parent->color;
18 node->parent->color = Node::BLACK;
19 // Step 3
20 distantNephew->color = Node::BLACK;
21 return;

```


红黑树与 4 阶 B 树 (2-3-4 树) 的关系



红黑树是由德国计算机科学家 [Rudolf Bayer](#) 在 1972 年从 B 树上改进过来的，红黑树在当时被称作 "symmetric binary B-tree"，因此与 B 树有众多相似之处。比如红黑树与 4 阶 B 树每个簇（对于红黑树来说一个簇是一个非 NIL 黑色节点和它的两个子节点，对 B 树来说一个簇就是一个节点）的最大容量为 3 且最小填充量均为 1。因此我们甚至可以说红黑树与 4 阶 B 树 (2-3-4 树) 在结构上是等价的。

对这方面内容感兴趣的可以观看 [从 2-3-4 树的角度学习理解红黑树 \(视频\)](#) 进行学习。

虽然二者在结构上是等价的，但这并不意味着二者可以互相取代或者在所有情况下都可以互换使用。最显然的例子就是数据库的索引，由于 B 树不存在旋转操作，因此其所有节点的存储位置都是可以确定的，这种结构对于不区分堆栈的磁盘来说显然比红黑树动态分配节点存储空间要更加合适。另外一点就是由于 B 树/B+ 树内储存的数据都是连续的，对于有着大量连续查询需求的数据库来说更加友好。而对于小数据量随机插入/查询的需求，由于 B 树的每个节点都存储了若干条记录，因此发生 cache miss 时就需要将整个节点的所有数据读入缓存中，在这些情况下 BST（红黑树，AVL，Splay 等）则反而会优于 B 树/B+ 树。对这方面内容感兴趣的读者可以去阅读一下 [为什么 rust 中的 Map 使用的是 B 树而不是像其他主流语言一样使用红黑树](#)。

红黑树在实际工程项目中的使用

由于红黑树是目前主流工业界综合效率最高的内存型平衡树，其在实际的工程项目中有着广泛的使用，这里列举几个实际的使用案例并给出相应的源码链接，以便读者进行对比学习。

Linux

源码：

- [linux/lib/rbtree.c](#)

Linux 中的红黑树所有操作均使用循环迭代进行实现，保证效率的同时又增加了大量的注释来保证代码可读性，十分建议读者阅读学习。Linux 内核中的红黑树使用非常广泛，这里仅列举几个经典案例。

[CFS 非实时任务调度](#)

Linux 的稳定内核版本在 2.6.24 之后，使用了新的调度程序 CFS，所有非实时可运行进程都以虚拟运行时间为键值用一棵红黑树进行维护，以完成更公平高效地调度所有任务。CFS 弃用 active/expired 数组和动态计算优先级，不再跟踪任务的睡眠时间和区别是否交互任务，而是在调度中采用基于时间计算键值的红黑树来选取下一个任务，根据所有任务占用 CPU 时间的状态来确定调度任务优先级。

[epoll](#)

epoll 全称 event poll，是 Linux 内核实现 IO 多路复用 (IO multiplexing) 的一个实现，是原先 poll/select 的改进版。Linux 中 epoll 的实现选择使用红黑树来储存文件描述符。

Nginx

源码：

- [nginx/src/core/nginx_rbtrees.h](#)
- [nginx/src/core/nginx_rbtrees.c](#)

nginx 中的用户态定时器是通过红黑树实现的。在 nginx 中，所有 timer 节点都由一棵红黑树进行维护，在 worker 进程的每一次循环中都会调用 `ngx_process_events_and_timers` 函数，在该函数中就会调用处理定时器的函数 `ngx_event_expire_timers`，每次该函数都不断的从红黑树中取出时间值最小的，查看他们是否已经超时，然后执行他们的函数，直到取出的节点的时间没有超时为止。

关于 nginx 中红黑树的源码分析公开资源很多，读者可以自行查找学习。

STL

源码：

- GNU libstdc++
 - [libstdc++-v3/include/bits/stl_tree.h](#)
 - [libstdc++-v3/src/c++98/tree.cc](#)
- LLVM libcxx
 - [libcxx/include/_tree](#)
- Microsoft STL
 - [stl/inc/xtree](#)

大多数 STL 中的 `std::map` 和 `std::set` 的内部数据结构就是一棵红黑树（例如上面提到的这些）。不过值得注意的是，这些红黑树（包括可能有读者用过的 `std::_Rb_tree`）都不是 C++ 标准，虽然部分竞赛（例如 NOIP）并未明令禁止这类数据结构，但还是应当注意这类标准库中的非标准实现不应该在工程项目中直接使用。

由于 STL 的特殊性，其中大多数实现的代码可读性都不高，因此并不建议读者使用 STL 学习红黑树。

OpenJDK

源码：

- [java.util.TreeMap<K, V>](#)
- [java.util.TreeSet<K, V>](#)
- [java.util.HashMap<K, V>](#)

JDK 中的 `TreeMap` 和 `TreeSet` 都是使用红黑树作为底层数据结构的。同时在 JDK 1.8 之后 `HashMap` 内部哈希表中每个表项的链表长度超过 8 时也会自动转变为红黑树以提升查找效率。

笔者认为，JDK 中的红黑树实现是主流红黑树实现中可读性最高的，本文提供的参考代码很大程度上借鉴了 JDK 中 `TreeMap` 的实现，因此也建议读者阅读学习 JDK 中 `TreeMap` 的实现。

参考代码

下面的代码是用红黑树实现的 `Map`，即有序不可重映射：

► 完整代码

其他资料

- [Red-Black Tree - Wikipedia](#)
- [Red-Black Tree Visualization](#)

-
1. https://en.wikipedia.org/wiki/Red-black_tree#cite_note-Cormen2009-18 ↩
 2. https://en.wikipedia.org/wiki/Red-black_tree#cite_note-Mehlhorn2008-17 ↩
 3. https://en.wikipedia.org/wiki/Red-black_tree#cite_note-Algs4-16: 432–447 ↩
-

本页面最近更新：2025/1/13 11:25:16, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [0x03A6](#), [abc1763613206](#), [auuuu4](#), [CCXXXI](#), [Conless](#), [Enter-tainer](#), [fanenr](#), [happyZYM](#), [hsfzLZH1](#), [iamtwz](#), [LeverImmy](#), [leverimmy](#), [LhcfI](#), [Marcythm](#), [Rlvance](#), [Tiphereth-A](#), [trudbot](#), [Xeniume](#), [Xeonacid](#), [yuhuoji](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用